Operating Systems — Practice Midterm Test

MAIN TEST · Multiple choice · Weeks 1–6 (Chapters I–III, Practice 1, labs) · 65 questions × 1 mark = 65 marks · Suggested time: 70 minutes

Name: _____	**Student ID:** _____	**Score:** _____ / 65

Instructions. Choose the **one best** answer and circle the letter. Some questions need a short calculation: use the margin. The answer key, with the working for every calculation, is on the last pages.

Chapter I — Introduction to OS and System Programming

1. Which statement best defines an **operating system**?

A. A program that only lets you browse the Internet
B. Software that manages hardware and software resources, provides common services for programs, and acts as an intermediary between users and hardware
C. The physical CPU and memory of a computer
D. A programming language used to write applications

2. Which is **not** one of the five main roles of an OS?

A. Process management
B. Memory management
C. Device management
D. Designing web pages

3. Which OS component interacts **directly with the hardware**?

A. Kernel
B. Shell
C. File system
D. Text editor

4. Bash and PowerShell are examples of:

A. Kernels
B. File systems
C. Shells
D. Device drivers

5. NTFS and FAT32 are examples of:

A. Shells
B. Device drivers
C. Kernels
D. File systems

6. Which OS type processes data with **strict timing constraints** (for example VxWorks, QNX)?

A. Batch
B. Real-time
C. Mobile
D. Distributed

7. Which OS type executes jobs in groups **without user interaction**?

A. Batch
B. Real-time
C. Embedded
D. Multi-user

8. Which OS type manages networked computers **as a single system** (for example Amoeba, Plan 9)?

A. Multi-user
B. Embedded
C. Distributed
D. Single-user

9. Which statement about kernels is correct?

A. Linux is a microkernel and QNX is monolithic
B. Linux is monolithic (services inside the kernel); QNX is a microkernel (services in user space)
C. Both Linux and QNX are monolithic
D. Neither Linux nor QNX has a kernel

10. What is a **system call**?

A. A function that exists only in Python
B. A keyboard interrupt
C. A command typed in the shell
D. The way a program asks the kernel to do something it cannot do itself

11. Which of these is a system call named in the slides?

A. `fork()`
B. `print()`
C. `len()`
D. `sort()`

12. Which is an example of **system** programming?

A. A web browser
B. A video game
C. A device driver
D. Office software

13. Which comparison of Python and C is correct?

A. Python is compiled and faster than C
B. C is compiled and faster with direct hardware access; Python is interpreted and slower
C. They have identical speed and portability
D. C programs never need recompiling for another OS

14. Which languages are typical for kernels and drivers?

A. JavaScript
B. C#
C. HTML
D. C, C++, Rust

Chapter II — Process and Thread Management

15. What is a **process**?

A. A file stored on disk
B. A thread
C. An instance of a program in execution
D. A system call

16. A process is made of:

A. Only its source code
B. Only its file name
C. Program code, its current activity (program counter, registers) and its resources (memory, files, CPU state)
D. Only the CPU

17. A process could run, but the CPU is busy with another process. Which state is it in?

A. New
B. Ready
C. Waiting
D. Terminated

18. A running process asks to read from the disk. Which state does it move to?

A. Ready
B. New
C. Terminated
D. Waiting

19. The disk read finishes. Where does the waiting process go next?

A. Straight to running
B. Terminated
C. Ready
D. New

20. What is a **zombie** process?

A. A running child whose parent has died
B. A terminated process that still has an entry in the process table
C. A service that starts at boot
D. A foreground process

21. What is an **orphan** process?

A. A terminated process with a table entry
B. A daemon
C. A child whose parent has terminated
D. A kernel thread

22. What is a **daemon**?

A. A foreground process
B. A zombie
C. A child process
D. A background service that starts at boot

23. What is a **thread**?

A. A process with its own isolated memory
B. A file
C. A program on disk
D. The smallest unit of execution within a process, sharing the process's memory and resources

24. Why is a thread context switch faster than a process context switch?

A. Threads have more memory
B. Threads are separate processes
C. Threads share the address space, so the page tables do not change
D. Threads do not use the CPU

25. How do two **processes** normally communicate?

A. Through shared global variables, automatically
B. Through IPC (pipes, shared memory, message queues)
C. They cannot communicate at all
D. Through page faults

26. What is true about **user-level** threads?

A. They are managed by the OS directly
B. They are always slower than kernel threads
C. They are the same as hybrid threads
D. They are fast but not visible to the OS; if one blocks, the whole program can stall

27. What is the slide example of **kernel-level** threads?

A. Game-engine threads managing rendering
B. A web server handling multiple client requests
C. Solaris
D. MS-DOS

28. Which system is the slide example of **hybrid** threads?

A. MS-DOS
B. Windows 95
C. Solaris
D. Amoeba

29. A CPU-heavy calculation should use all CPU cores. Which approach fits best?

A. Threading only
B. Multiprocessing
C. One single thread
D. Zombie processes

30. What is the difference between concurrency and parallelism?

A. Concurrency = tasks in progress taking turns; parallelism = tasks running at the same instant on different cores
B. They mean exactly the same thing
C. Parallelism = taking turns; concurrency = at the same instant
D. Concurrency needs several computers

Practice 1 — Process or Thread?

31. A word processor freezes after opening a large file, but the OS and other apps keep working. What is the problem?

A. The process has terminated
B. The OS kernel crashed
C. It is a zombie process
D. A thread-level problem: the UI thread is blocked

32. What design keeps the word processor responsive?

A. One thread for the UI and another thread for loading the file
B. Only one thread for everything
C. Closing the OS while loading
D. More zombie processes

33. The process table fills with Z entries. What has the parent process failed to do?

A. Call `fork()`
B. Call `exec()`
C. Call `wait()` or `waitpid()`
D. Call `open()`

34. What is the real damage caused by many zombies?

A. They use a lot of CPU
B. The process table is finite; when it is full, no new process can start
C. They use a lot of memory
D. They delete files

35. Why does Chrome run each tab as a separate **process**?

A. Fault isolation and security
B. To use less memory
C. Because threads cannot show web pages
D. To make tabs share memory

36. What is the main **disadvantage** of Chrome's design?

A. Less security
B. Tabs share memory
C. No isolation between tabs
D. Resource overhead: separate address spaces, page tables and costlier context switches

37. Design A (process per request) survives a crash that kills design B (thread per request). Why?

A. Threads are slower
B. Processes have no memory
C. In B, all threads share one address space, so one bad pointer kills the whole process
D. The OS restarts crashed threads automatically

38. Which design gives both performance and isolation?

A. A single thread for all requests
B. A process pool where each process runs a thread pool
C. One process per byte of data
D. A pool of zombie processes

39. Two threads each run `counter++` one million times. The result is lower than 2,000,000 and different every run. What is this?

A. A race condition
B. A deadlock
C. A page fault
D. A zombie

40. Whose fault is the lost-increment problem?

A. The CPU
B. The OS scheduler
C. The threads
D. The programmer, who did not use a mutex, semaphore or atomic operation

41. Which item is **private** to each thread?

A. Heap
B. Global variables
C. Stack
D. Open file descriptors

42. Which item is **shared** between threads of one process?

A. Registers
B. Program counter
C. Stack
D. Heap

Chapter III — Memory Management

43. What are the four goals of memory management?

A. Allocation, protection, sharing, optimization
B. Encryption, hashing, salting, backup
C. Scheduling, printing, booting, logging
D. Reading, writing, deleting, renaming

44. Which is the **fastest** level of the memory hierarchy?

A. RAM
B. Cache
C. CPU registers
D. SSD

45. Going **down** the memory hierarchy, memory becomes:

A. Faster, smaller and more expensive
B. Slower, bigger and cheaper per bit
C. Exactly the same
D. Faster, bigger and cheaper

46. **Spatial** locality means:

A. The same data is used again soon
B. Data is accessed in random order
C. Data is stored on disk
D. Data near recently used data is likely to be used soon

47. What is the main problem of **fixed** partitioning?

A. Internal fragmentation
B. External fragmentation only
C. It needs a page table
D. It cannot hold any process

48. What is the main problem of **dynamic** partitioning?

A. Internal fragmentation
B. External fragmentation, which needs periodic compaction
C. It wastes space inside each page
D. It needs segment tables

49. In paging, which statement is correct?

A. A page is physical and a frame is logical
B. Pages and frames have different sizes
C. A page is a block of logical memory; a frame is a same-size block of physical memory
D. Pages and frames are the same thing

50. What does paging eliminate?

A. All fragmentation
B. Page tables
C. Internal fragmentation in the last page
D. External fragmentation

51. Which statement describes **segmentation**?

A. Variable-size segments matching program structure (code, data, stack), which can cause external fragmentation
B. Fixed-size blocks with no external fragmentation
C. Power-of-2 blocks that split and merge
D. Pre-allocated slabs for kernel objects

52. Page size = 1024 bytes. Logical address = 3000. What are the page number and offset?

A. Page 3, offset 0
B. Page 2, offset 976
C. Page 2, offset 952
D. Page 1, offset 1976

53. Continuing: the page table maps page 2 → frame 5. What is the physical address?

A. 3000
B. 6072
C. 5952
D. 7072

54. What is a **page fault**?

A. A fatal error that crashes the program
B. A bug in the program
C. A disk failure
D. A normal signal that the page is not in memory yet, so the OS loads it

55. The **LRU** algorithm replaces:

A. The least recently used page
B. The oldest loaded page
C. A random page
D. The most frequently used page

56. Which statement about the **Optimal** algorithm is correct?

A. It replaces the oldest page
B. It replaces a random page
C. It needs knowledge of the future, so it is a theoretical benchmark
D. It replaces the most used page

57. Reference string **1, 2, 3, 4, 1, 2** with **3 frames** using **FIFO**. How many page faults?

A. 4
B. 6
C. 5
D. 3

58. Buddy system: 1024 KB free, a process requests **60 KB**. Which block size is allocated?

A. 60 KB
B. 128 KB
C. 32 KB
D. 64 KB

59. What is **slab allocation**?

A. Pre-allocated slabs for frequently used kernel objects
B. Power-of-2 blocks
C. Variable-size segments
D. Fixed partitions set at boot

60. What is **virtual memory**?

A. Extra RAM chips added to the motherboard
B. The CPU cache
C. Part of secondary storage used to give the illusion of more main memory, raising the degree of multiprogramming
D. GPU memory

Labs — Reading the Code

61. In the week 1 lab, what does `os.getpid()` return?

A. The parent's program name
B. The process ID of the running process
C. The CPU usage
D. The list of files

62. `physical_memory = [None, 'OS', None, 'AppX', None, None, 'OS', None]`. How many frames are free?

A. 3
B. 8
C. 5
D. 4

63. In the paging lab, what happens if the line `available_frames.remove(physical_frame_num)` is commented out?

A. Several pages can get the same frame and overwrite each other
B. The program becomes faster and still correct
C. No pages are loaded at all
D. The program uses segmentation instead

64. What changes when `random.seed(42)` is added to the paging lab?

A. The page table is different every run
B. The program raises an error
C. No free frames remain
D. The same page table is produced every run (reproducible)

65. In the week 3 dashboard, why is `root.after(1000, update)` used instead of a `while` loop with `sleep`?

A. It freezes the window on purpose
B. It schedules `update` every second without blocking the UI thread
C. It creates a new process each second
D. It kills the UI thread

Answer Key

Q	Ans	Why
1	B	Manages resources, provides services, intermediary between users and hardware.
2	D	Roles: process, memory, file system, device management, user interface.
3	A	The kernel is the core that talks to hardware.
4	C	A shell takes commands and executes them.
5	D	File systems organize and manage data storage.
6	B	RTOS = strict timing constraints.
7	A	Batch (IBM OS) runs jobs without user interaction.
8	C	Distributed OS = many computers acting as one system.
9	B	Monolithic = services inside the kernel; microkernel = minimal kernel, services outside.
10	D	Programs request kernel services through system calls.
11	A	`fork()`, `exec()`, `open()` are the slide examples.
12	C	Drivers manage hardware; the others solve end-user problems.
13	B	C: compiled, fast, low level. Python: interpreted, slower, easier.
14	D	C, C++, Rust (and Assembly).
15	C	A program is a file; a process is that program running.
16	C	Code + current activity + resources.
17	B	Ready = could run, CPU busy.
18	D	Running → waiting when it needs I/O.
19	C	Waiting → ready (not straight to running).
20	B	Dead, but the parent has not collected the exit status.
21	C	Still alive; its parent is gone.
22	D	A background service that starts at boot.
23	D	Smallest unit of execution, shares memory.
24	C	Only registers, stack pointer and PC are swapped; no page-table switch.
25	B	Processes are isolated, so they need IPC.
26	D	Invisible to the OS; one block can stall all.
27	B	Game engine is the user-level example; Solaris is hybrid.
28	C	Solaris = hybrid.
29	B	Multiprocessing gives true parallelism for CPU-bound work.
30	A	Concurrency = taking turns; parallelism = same instant on different cores.
31	D	The process is alive; the UI thread is blocked on I/O or long work.
32	A	Separate UI and worker threads.
33	C	The parent never reaps its children (or use a `SIGCHLD` handler).
34	B	Zombies use no CPU/memory, but table slots run out.
35	A	A crash or exploit in one tab cannot touch the others.

Q	Ans	Why
36	D	Every process has its own address space and kernel structures.
37	C	Shared address space = shared fate.
38	B	Hybrid: process pool + thread pool (modern web servers).
39	A	`counter++` is load → increment → store, not atomic.
40	D	The OS offers mutexes, semaphores and atomics; the programmer must use them.
41	C	Private: stack, registers, PC, thread-local storage.
42	D	Shared: code, globals (data segment), heap, open files, cwd, signal handlers.
43	A	Allocation, protection, sharing, optimization.
44	C	Registers → cache → RAM → secondary → tertiary.
45	B	Slower → bigger → cheaper per bit.
46	D	Temporal = same data again; spatial = nearby data.
47	A	Waste inside a fixed partition.
48	B	Gaps between blocks; compaction merges them.
49	C	Same size; the page table maps page → frame.
50	D	Any page fits any frame, so no gaps between blocks. Small internal waste remains.
51	A	Segment table holds base and length.
52	C	3000 ÷ 1024 = 2 (whole part); 3000 − 2048 = 952.
53	B	5 × 1024 + 952 = 5120 + 952 = 6072.
54	D	A page fault is not an error.
55	A	LRU looks at the past use.
56	C	Optimal replaces the page not needed for the longest time.
57	B	1F 2F 3F → [1 2 3]; 4F (out 1) → [4 2 3]; 1F (out 2) → [4 1 3]; 2F (out 3) → [4 1 2]. 6 faults.
58	D	Smallest power of 2 ≥ 60 KB: 1024 → 512 → 256 → 128 → 64.
59	A	Slabs give fast, fragmentation-free allocation of kernel objects.
60	C	Usually implemented with demand paging.
61	B	PID = process ID.
62	C	Frames 0, 2, 4, 5, 7 are `None`.
63	A	Nothing marks the frame as taken.
64	D	A fixed seed makes the random sequence reproducible.
65	B	A `sleep` loop in the UI thread would freeze the window (Practice 1, scenario 1).

Scoring guide: 59–65 excellent · 49–58 good, review the chapters you missed · below 49 re-read the study guide, then take the EXTRA practice bank.